

Prompt input/output reference — every LLM call with a worked example

The complete set of LLM calls in the current design, in pipeline order, each with **what we send in**, **what we get back** (the structured-output schema), and a **concrete example**. All generation calls read the **raw idea-card text** — no generation signals. Output is always a typed pydantic schema passed to the gateway as structured output, never a JSON block in the prompt.

A running example ticket is used throughout:

IDMT-19761 — "Enabling Real-Time Quote Automation for Enterprise Accounts." Sales Operations needs a real-time CPQ integration to cut the enterprise quote cycle from 5 days to same-day. Discounts above 20% require VP approval. Accepted quotes must hand off to order fulfilment; Deal Desk SLA target is 4 hours.

1. Condense (LLM ×1, per ticket)

Turns the raw ticket packet into structured summary fields for retrieval/routing, and passes the raw text through for generation.

Prompt: `condense/summary` **Input:** `ticket_id`, `consolidated_text` (the ~24k-token raw idea-card text). **Output schema:** `SummaryFields { generatedSummary, businessProblem, businessCapability, keyTerms[], stakeholders[], systemsAndProducts[] }`

Example output:

```
{
  "generatedSummary": "Sales Operations needs a real-time CPQ integration to cut the enterprise quote cycle from five days to same-day, with automated discount approval and a handoff to order fulfilment.",
  "businessProblem": "Manual quoting delays enterprise deal closures by 3-5 days.",
  "businessCapability": "Automated real-time quote generation for enterprise accounts.",
  "keyTerms": ["CPQ", "quoting", "enterprise", "Salesforce", "Deal Desk"],
  "stakeholders": ["Sales Ops", "IT", "Finance"],
  "systemsAndProducts": ["Salesforce CPQ", "Oracle ERP", "Deal Desk Portal"]
}
```

2. Value Stream Selection (LLM ×1, per ticket)

Picks the relevant Value Streams from the full governed set, using the new ticket plus similar past tickets as precedent.

Prompt: `value_stream/selection_evidence_recall` (the winning "evidence" mode) **Input:**

- `query_for_prompt` — the new ticket's **raw idea-card text** (the lever; the summary is only the retrieval query)
- `requested_final_output_count` — exact number to return (default 10)
- `historic_evidence` — the **6 similar past tickets**, each as its summary + the Value Streams it was tagged with
- `candidate_blocks` — **all 50 governed Value Streams**, one compact block each

Candidate block format (one per Value Stream, all 50 sent):

```
Candidate: Configure, Price and Quote
entity_id: VSR-0042
description: Initiate, price and issue enterprise quotes.
category: Sales and Enrollment
trigger: A qualified enterprise opportunity needs a quote.
value: Faster, accurate quotes that shorten the sales cycle.
assumptions: Pricing is sourced from the master pricing system.
```

Historic evidence block:

```
SIMILAR PAST TICKETS (evidence – the value streams these were tagged with):
- IDMT-17432: CPQ enhancements for the large Deal Desk; automated pricing configuration for enterprise tiers.
  -> tagged value streams: Configure, Price and Quote (VSR-0042), Order Management (VSR-0017)
- IDMT-16890: Order fulfilment integration with CPQ; quote-to-order handoff latency reduction.
  -> tagged value streams: Order Management (VSR-0017)
```

Output schema: `recommendations[] { valueStreamId, valueStreamName, confidence (0-100), supportType (direct|implied), reason, sourceTickets[] (implied only) }`

Example output:

```
{
  "recommendations": [
    { "valueStreamId": "VSR-0042", "valueStreamName": "Configure, Price and Quote", "confidence": 91,
      "supportType": "direct", "reason": "Ticket explicitly targets CPQ automation for enterprise quoting.", "sourceTickets": [] },
    { "valueStreamId": "VSR-0017", "valueStreamName": "Order Management", "confidence": 74,
      "supportType": "implied", "reason": "Quote-to-order handoff implied by the fulfilment requirement.", "sourceTickets": ["IDMT-16890"] }
  ]
}
```

(Human approval gate here — the SME confirms the Value Stream set before anything is generated.)

3. Stage Selection (LLM ×1, all Value Streams batched)

For every approved Value Stream, picks which of its governed lifecycle stages the work touches — one batched call.

Prompt: theme/stage_selection **Input:** ticket_context (raw idea-card text) + value_streams — each approved VS with its own candidate stages.

```
## Approved value streams (each with its own candidate stages)
### Value Stream VSR-0042 – Configure, Price and Quote
[1] Opportunity to Quote (VSS-0042-01)
description: Initiate and price an enterprise quote. entrance: opportunity qualified | exit: quote issued
[2] Quote to Order (VSS-0042-02)
description: Convert an accepted quote into an order. entrance: quote accepted | exit: order created
...
```

Output schema: value_streams[] { valueStreamId, selectedStages[] { stageId, stageName, reason } }

Example output:

```
{
  "value_streams": [
    { "valueStreamId": "VSR-0042", "selectedStages": [
      { "stageId": "VSS-0042-01", "stageName": "Opportunity to Quote", "reason": "Ticket targets the quoting initiation workflow." },
      { "stageId": "VSS-0042-02", "stageName": "Quote to Order", "reason": "Quote-to-order handoff is in scope." } ] }
  ]
}
```

A stage placed under the wrong VS is salvaged to its owner (ids are globally unique). An empty list means "take the whole governed list for the architect to trim."

4. Description BODY (LLM ×1, per ticket, VS-agnostic)

Writes the shared narrative body of the Theme description.

Prompt: theme/description_body **Input:** ticket_context (raw idea-card text). **Output schema:** { text } (the shared narrative body).

Example output:

```
This theme automates enterprise quoting by integrating Salesforce CPQ with real-time pricing, cutting the quote cycle from five days to same-day. Discounts above 20% route to VP approval, and accepted quotes hand off to order fulfilment within the Deal Desk's 4-hour SLA.
...
```

5. Description FRAMING (LLM ×1, all Value Streams batched)

A short per-Value-Stream intro paragraph, batched in one call.

Prompt: theme/description_framing **Input:** ticket_context (raw) + value_streams (each approved VS: id, name, description, value proposition). **Output schema:** framings[] { valueStreamId, text }

Example output:

```
{
  "framings": [
    { "valueStreamId": "VSR-0042", "text": "Within Configure, Price and Quote, this initiative makes enterprise quoting real-time and rule-driven." },
    { "valueStreamId": "VSR-0017", "text": "For Order Management, it ensures accepted quotes flow cleanly into fulfilment." }
  ]
}
```

Final per-VS description = its framing paragraph + the shared body (concatenated, deterministically).

6. Business Needs (LLM ×1 per Value Stream)

Writes the consolidated Business Needs document for one Value Stream's selected stages.

Prompt: theme/business_needs **Input:** the approved VS (id, name, description, valueProposition), its selected_stages (stageId, stageName, stageDescription), and the raw ticket_context . **Output schema:** { text } — one consolidated document, a "Value Stage:" block per selected stage.

Example output:

```
Value Stage: Opportunity to Quote

Business Product Feature: Real-Time Quoting
1. Sales Operations requires real-time quote generation to replace the current 3–5 day manual cycle.
   Dependency: Salesforce CPQ and the Oracle pricing API.
   Business Rule: Discounts above 20% require VP approval.

Value Stage: Quote to Order

Business Product Feature: Order Handoff
1. Accepted quotes must hand off to order fulfilment within the Deal Desk 4-hour SLA.
```

7. Capabilities (L3) (LLM ×1, ALL Value Streams merged)

One merged call selects the applicable L3 capabilities for every approved Value Stream's selected stages. **L2 is derived 1-1 from the selected L3 — no separate L2 call.**

Prompt: theme/capability_selection_merged **Input:** ticket_context (raw) + value_streams, grouped **Value Stream → Stage → governed candidate L3:**

```
## Approved value streams, each with its selected stages and each stage's own candidate L3
### Value Stream VSR-0042 – Configure, Price and Quote
description: Initiate, price and issue enterprise quotes.

### Stage VSS-0042-01
[1] Opportunity to Quote (VSS-0042-01)
description: Initiate and price an enterprise quote. entrance: opportunity qualified | exit: quote issued
Candidate L3 capabilities (choose by id; each shows its parent L2):
- L3-0042-0011 | Real-Time Pricing Engine Integration – live CPQ pricing feed [tier: core] (L2: Pricing and Discounting)
- L3-0042-0012 | Automated Discount Approval Workflow – routes discounts for approval (L2: Approval and Governance)

### Value Stream VSR-0017 – Order Management
...
### Stage VSS-0017-01
...
- L3-0017-0011 | Order Intake Automation – captures order data (L2: Order Management)
```

Output schema: stages[] { stageId, capabilities[] { capabilityId, capabilityName, reason } } — keyed by stageId, re-attributed to each Value Stream afterward.

Example output:

```
{
  "stages": [
    { "stageId": "VSS-0042-01", "capabilities": [
      { "capabilityId": "L3-0042-0011", "capabilityName": "Real-Time Pricing Engine Integration", "reason": "Live CPQ pricing feed from Oracle ERP." },
      { "capabilityId": "L3-0042-0012", "capabilityName": "Automated Discount Approval Workflow", "reason": "Discounts above 20% require VP approval." } ] },
    { "stageId": "VSS-0017-01", "capabilities": [
      { "capabilityId": "L3-0017-0011", "capabilityName": "Order Intake Automation", "reason": "Captures the order on quote acceptance." } ] }
  ]
}
```

*Strict stage isolation + a deterministic salvage step keep each L3 under its owning stage. **L2** = the unique parent L2s of the selected L3, derived deterministically per Value Stream.*

Final Theme package (deterministic — no LLM)

Assembled per approved Value Stream:

```
{
  "themeTitle": "Enabling Real-Time Quote Automation -- Configure, Price and Quote",
  "themeDescription": "<framing + body>",
  "selectedStages": [ { "stageId": "VSS-0042-01", "stageName": "Opportunity to Quote", "reason":
"..."} ],
  "businessNeeds": "<consolidated text>",
  "l3Capabilities": [ { "stageId": "VSS-0042-01", "capabilities": [ ... ] } ],
  "l2Capabilities": [ { "stageId": "VSS-0042-01", "capabilities": [ { "capabilityId": "L2-0042-001", "name": "Pricing and Discounting" } ] } ]
}
```

Evaluation judges (eval-only, reference-free)

These never run in production. They judge a generated artifact against its **source** (the idea card), not against ground truth. Claim extraction runs once and its claims feed both faithfulness and correctness.

J1. Claim extraction • `judges/claim_extraction`

Input: the generated `text`. **Output:** `ClaimList { claims[] }`.

```
{ "claims": [
  "The integration cuts the enterprise quote cycle from five days to same-day",
  "Discounts above 20% require VP approval",
  "Accepted quotes hand off to order fulfilment" ] }
```

J2. Faithfulness / hallucination • `judges/faithfulness`

Input: `source` + the extracted `claims`. **Output:** `FaithfulnessResult { claims[] { claim, supported } }`.
 $\text{faithfulness} = \text{supported} / \text{total}$; $\text{hallucination} = 1 - \text{faithfulness}$.

```
{ "claims": [
  { "claim": "...quote cycle from five days to same-day", "supported": true },
  { "claim": "Discounts above 20% require VP approval", "supported": true },
  { "claim": "Launches in Q3", "supported": false } ] } // -> faithfulness 0.67
```

J3. Correctness • `judges/correctness`

Input: `source` + `claims`. **Output:** `CorrectnessResult { claims[] { claim, correct } }`. Stricter than supported — catches distorted detail (wrong number/scope/direction). $\text{correct} / \text{total}$.

```
{ "claims": [
  { "claim": "Discounts above 20% require VP approval", "correct": true },
  { "claim": "All discounts require VP approval", "correct": false } ] } // scope distorted
```

J4. Coverage • `judges/coverage`

Input: `source` + the generated `description`. **Output:** `CoverageResult { facts[] { fact, covered } }`.
Extracts the source's key facts, marks each reflected. `covered / total`.

```
{ "facts": [
  { "fact": "Cut quote cycle from 5 days to same-day", "covered": true },
  { "fact": "Hand off accepted quotes to fulfilment", "covered": false } ] } // -> coverage 0.5
```

J5. Stage usage (Business Needs only) • `judges/stage_usage`

Input: the selected `stages` (with scope) + the `business_needs` document. **Output:** `StageUsageResult { stages[] { stageId, addressed, aligned, note } }`. `usage = addressed / selected`; `alignment = aligned / addressed`.

```
{ "stages": [
  { "stageId": "VSS-0042-01", "addressed": true, "aligned": true, "note": "Real-time quoting needs are in scope." },
  { "stageId": "VSS-0042-02", "addressed": true, "aligned": false, "note": "Order-creation need belongs to a later stage." } ] }
```